Check for updates

The International Journal of
Robotics Research
32(8) 873–888
© The Author(s) 2013
Reprints and permissions:
sagepub.co.uk/journalsPermissions.nav
DOI: 10.1177/0278364913488427
ijr.sagepub.com
SAGE

Optimizing waypoints for monitoring spatiotemporal phenomena

Jonathan Binney[1], Andreas Krause[2] and Gaurav S. Sukhatme[1]

Abstract

We introduce a graph-based informative path planning algorithm for a mobile robot which explicitly handles time. The objective function must be submodular in the samples taken by the robot, and the samples obtained are allowed to depend on the time at which the robot visits each location. Using a submodular objective function allows our algorithm to handle problems with diminishing returns, e.g. the case when taking a sample provides less utility when other nearby points have already been sampled. We give a formal description of this framework wherein an objective function that maps the path of the robot to the set of samples taken is defined. We also show how this framework can handle the case in which the robot takes samples along the edges of the graph. A proof of the approximation guarantee for the algorithm is given. Finally, quantitative results are shown for three problems: one simple example with a known Gaussian process model, one simulated example for an underwater robot planning problem using data from a well-known ocean modeling system, and one field experiment using an autonomous surface vehicle (ASV) measuring wireless signal strength on a lake.

Keywords

marine robotics, mapping, sensor networks

1. Introduction

We address the problem of informative path planning (IPP) for a mobile robot. This is the problem of choosing a path for a robot that maximizes the sensor information obtained about an underlying field of interest. IPP is especially useful for robots in environmental applications: for example, an unmanned underwater vehicle (UUV) being used to monitor water temperature in a region of the ocean, or an unmanned aerial vehicle (UAV) being used to measure air quality.

In order to use mobile robots for these types of tasks, a path must be chosen for them to execute. Once the path is chosen, it is executed by the robot, and the measurements that the robot acquires along the path are used to improve predictions of the quantity being modeled. Although a human expert with domain knowledge could manually specify a path for the robot, there are several advantages to being able to autonomously plan such a path:

- Human experts may not always be available.
- As robots get cheaper, in some cases the cost of the human operator may be the limiting factor.
- Some robots have limited communications (for instance underwater vehicles). They may need to re-plan on their own.
- When complex probabilistic models of the environment are used, planning algorithms may be able to take advantage of structure in the model that is not intuitive to a human.

Before a planning algorithm can be used, however, the problem must be specified mathematically. There are many different possible ways to specify the problem, and several important questions must be answered:

- What is the probabilistic model of the quantity being measured?
- What is the objective function that describes how "informative" a set of measurement locations are?
- What constraints are placed on the path?

For this paper, we will use a Gaussian process (GP) model of the quantity of interest. Our objective function will be the average reduction in variance of the GP model at the locations we care about. The path will be described as a sequence of edges along a pre-defined waypoint graph, with

[1]University of Southern California, Los Angeles, CA, USA
[2]ETH Zurich, Zurich, Switzerland

Corresponding author:
Jonathan Binney, University of Southern California, Ronald Tutor Hall, 3710 South McClintock Avenue, Los Angeles, CA 90089-2900, USA.
Email: binney@usc.edu

fixed start and end locations and a maximum allowed cost. We give a formal definition of the problem in Section 3.

The contribution of this paper is an algorithm called generalized recursive greedy (GRG) that can explicitly handle two more complicated variants of IPP. The first variant is that of time-varying fields, in which the usefulness of taking a measurement at a particular location depends explicitly on the time at which the measurement is taken. The second is the case of edge-based samples, which is the case when the robot samples while moving along the path, not just at the waypoints of the graph. We give a rigorous description of the algorithm, and prove an approximation guarantee for it. We also give quantitative results for three different example problems: a small toy problem, a realistic environmental monitoring dataset from a widely used ocean modeling system, and a field experiment in which we deploy an autonomous surface vessel (ASV) to monitor the strength of a wireless signal across the surface of a lake.

2. Related work

2.1. Connection to other path planning problems

The difference between IPP and other path planning problems lies in the objective function. Many path planning problems attempt to find the shortest path between two points. This kind of problem, in which the overall solution can be decomposed into individually solving shortest path subproblems, can typically be solved in polynomial time by dynamic programming approaches. Often, heuristics are employed to further speed up computation, resulting in algorithms such as A* (LaValle, 2006).

In IPP, however, the usefulness of the measurements taken at each part of the path are dependent on the locations of all of the other measurements taken along the path. For example, if the robot measures a location near the beginning of the path, re-visiting that location near the end of the path is probably not very useful. More formally, the objective functions for IPP are usually not modular. This characteristic of the problem rules out simple dynamic programming approaches. Instead, IPP has more in common with the objective functions used in problems such as robot search. In search, the value of a robot searching any given area may depend on whether it searches it at any other point along the path. The relationship between IPP and the robot search problem is discussed by Hollinger and Singh (2008).

Sometimes a seemingly complicated planning objective can be solved using an algorithm which does not plan far into the future. An example of this is the utility-based algorithm of Rooker and Birk (2007) for exploration under communication constraints. IPP could be addressed with a similar approach, but in the case of expensive air or sea robots being deployed to collect scientific data, it often makes sense to use a large amount of computation beforehand to plan over a longer time horizon. This is especially true when there is a hard constraint on the amount of time the robot has to complete its task.

A wheeled robot is used to build grid maps of gas concentrations in Lilienthal and Duckett (2004), employing both pre-defined paths and reactive control strategies. In contrast with that work, we focus on planning entire paths to collect the most useful data, rather than reactive behaviors. This means that our approach will generally be slower and more computationally intensive, but will have advantages when local minima are a problem.

GP models of the environment have been used in a number of contexts. They are used for gas concentration modeling with pre-defined robot paths in Stachniss et al. (2009), and for terrain modeling in Vasudevan et al. (2009). Both of those works explore GP modeling in more depth than we do here. We treat the GP as a black box which gives us an estimate of uncertainty based on measurement locations, but our focus is on the path planning problem.

The most common way to model uncertainty in AI planning problems is to use a Markov decision process (MDP) or partially observable Markov decision process (POMDP) (Russell and Norvig, 2002). In the case of IPP, a full POMDP would be needed because the values of the observations (measurements) are not known until they are taken. Because IPP involves reducing the uncertainty of a scalar field, the value of the field at all areas of interest must be modeled. This requires a large number of continuous variables, which leads to an even larger belief space, making using a general-purpose POMDP solver impractical for IPP.

2.2. Sensor placement and feature selection

Sensor placement is the problem of choosing a subset of a larger set of possible locations at which to place sensors, in order to get the most useful measurements. It is similar to IPP, but the sensor locations are not constrained to be on a path. The diminishing returns gained by placing multiple sensors at nearby locations can be captured by submodular objective functions. Mutual information is one such submodular objective function, and has been used for sensor placement (Krause et al., 2006).

Another problem similar to sensor placement is feature selection. This is the problem of choosing the most useful subset of features from a larger set. In feature selection problems, an assumption of monotonicity is often made. This is a less restrictive assumption than that of submodularity. Branch and bound approaches for optimal feature selection have been the subject of quite a bit of work (Narendra and Fukunaga, 1977; Somol et al., 2004). Unfortunately, adding a path length constraint makes the sensor placement/feature selection problem much more difficult.

2.3. IPP

Existing solutions to the informative path problem can be classified in terms of the restrictions applied to the objective function. One of the strongest restrictions is to require

that the objective function be a sum of a constant reward given for each individual node visited. This approach is taken by Zhang (2008), who uses an algorithm from Blum et al. (2007) to provide a constant factor approximation for the orienteering problem. The assumption that the objective function is the sum of an objective obtained at each node is quite limiting for IPP. It assumes that the reward obtained by visiting a given location does not depend on what other locations are visited. In reality, visiting a location often decreases the usefulness of visiting nearby locations. A linearity assumption is also made by Yilmaz et al. (2008), although they allow for the inclusion of a wide variety of constraints and objectives as part of a mixed-integer linear program.

The problem of planning a sequence of several paths all starting and ending at the same node was considered by Meliou et al. (2007), who described an algorithm for planning a sequence of paths, one for each timestep, all starting and ending at the same node (as in sensor network data gathering tours). In that work the planner used to find a path for each time step was treated as a black box; the algorithm we present in this paper could be used there.

Choosing a path which maximizes a submodular objective subject to constraints on path length is NP-hard. IPP with mutual information as the objective function using the recursive greedy approximation algorithm from Chekuri and Pal (2005) is studied by Singh et al. (2009a). This is the work most closely related to ours. Our contributions in this work include showing how to generalize the recursive greedy algorithm to solve IPP problems with time-varying objective functions and edge based samples, without significantly increasing the running time.

In some specific IPP problems, the objective function is not only submodular but also local (samples far from each other are almost independent). An efficient two level algorithm for this case based on a padded decomposition of the graph is presented by Singh et al. (2009b). In the same work, an extension to the algorithm and a multi-robot extension based on sequential allocation are studied.

In our previous work, we presented a method for doing IPP for scalar fields with edge-based reward by extending the algorithm from Chekuri and Pal (2005), but without a proof of the approximation guarantee. Further, Binney et al. (2010) did not deal with time-varying cases which we address here.

3. Problem statement

To describe the possible paths of the robot, we define a graph G in which the vertices are the possible waypoints which the robot can visit, and the edges represent movements that the robot can make from one waypoint to another. The job of the planner is to choose a path P for the robot, consisting of the sequence of edges which the robot will traverse. We fix the start waypoint s and end waypoint t, restricting the paths which the planner can choose. In the case of environmental monitoring for mobile robots, fixing the start and end points makes sense, because we want to be able to deploy and recover the robot at known locations.

We also constrain the path to a maximum length $\|P\| \leq B$. The cost of a path is defined as the sum of the lengths of the edges in the path, and the cost of each edge is set to the amount of time required for the robot to traverse that edge. Although in the general case these costs could depend on time, the examples in this paper all assume that the cost of an edge is unchanging.

We define a ground set of all possible measurement locations $\mathcal{V}$. The elements in this set depends on the exact variant of IPP which is being solved, which we describe more in Section 5. In the general case, for every possible spatial location at which the robot could possibly take a measurement while traversing any edge in the graph, there is one element in $\mathcal{V}$ for each possible time at which the sample could be taken. This is a finite set, because we discretize both space and time.

The path chosen by the planner implicitly determines the locations and (given an assumption of a known constant velocity) times at which the robot will take measurements. The constant velocity assumption makes sense for aquatic vehicles that are controlled by giving them a coarse set of waypoints to visit, but may not make sense for all robots.

To formalize this, we define a function $\mathcal{A} = h(P, \tau)$ which takes a path P and starting time τ, and returns the set $\mathcal{A} \subset \mathcal{V}$ of measurement locations/times that the robot would visit while executing the path. Finally, we choose an objective function $f(\mathcal{A})$ which defines the "informativeness" of taking measurements at a set $\mathcal{A}$ of locations. We describe this function in more detail in Section 4.

Given these definitions, IPP is the problem of finding a path P^* on a graph G, starting at waypoint s and ending at waypoint t, such that

$$P^* = \arg\max_P f(\, h(P, \tau)\,)$$

subject to $\|P\| < B$.

In previous IPP works, the set of sample locations have typically been assumed to be the same as the vertices in the graph G which describes the possible movements of the robot. As we will show in Section 5, this assumption is not true for some important variants of IPP. Adding the intermediate function $h(\,\cdot)$ allows us to handle some useful variants of the problem in a natural way, without significantly increasing the running time of the algorithm.

4. Objective functions for IPP

In order to fully define a specific IPP problem, we must choose a measure of "informativeness" to use for the objective function $f(\,\mathcal{A})$. Two common choices for the objective function are the mutual information of sensed and unsensed locations (Krause et al., 2006), and the reduction in variance

of all locations. A useful characteristic of mutual information, and in some cases variance reduction, is submodularity. Formally, submodularity requires that given two sets A and B, such that $A \subseteq B$, and an element s not in B,

$$f(A \cup \{s\}) - f(A) \geq f(B \cup \{s\}) - f(B) \quad (1)$$

Submodularity expresses the idea of "diminishing returns". Intuitively, Equation (1) means that the benefit obtained by adding an element s decreases as more values are included in the set. In the case of IPP, each element in the set is a time/location at which the robot can take a sample. As the robot collects more samples, each new sample gives less information about the underlying field.

Mutual information is always submodular, and monotonic for small sets of measured locations (Krause et al., 2006). For the results we present in Section 7, we use expected reduction of mean squared error (EMSE), or equivalently variance reduction. Although not always submodular, for some kinds of Gaussian models submodularity can be proved (Das and Kempe, 2008). In practice, we have found our algorithm to work quite well on this objective function.

4.1. Modeling the underlying field

To calculate the variance reduction of a set of samples, we need a model of the underlying field being monitored. We assume that the field exists in a D-dimensional space, and takes a scalar value at each location in that space. All D dimensions can be spatial, or one can be time. We model the possible values of the field as a multivariate Gaussian, where every location in the D-dimensional space is a random variable. In reality, we only care about the values at a discrete set of locations, described by the set of all possible measurement times/locations $\mathcal{V}$.

In order to calculate the mutual information of any two sets of samples, we need the covariance matrix of the prior distribution of the variables. We consider two cases: one where we have a large amount of past data for this field and another where we do not. If large amounts of past data for the scalar field are available, then it is possible to use the empirical covariance of the data as the covariance matrix of the prior, as done in Krause et al. (2008). This data could come from remote sensing, from past robot deployments, or from a static sensor network.

In the case where large amounts of past data are not available and the empirical covariance matrix cannot be calculated, a GP can be used (Rasmussen and Williams, 2006). First, a small amount of data are collected, typically by having the robot take a relatively small number of samples on a short, preprogrammed path. These samples are used to estimate the hyperparameters of the kernel function of the GP. The kernel function can then be used to create a kernel matrix, which is used in place of the empirical covariance. Here, we use the simple exponential kernel

$$k(x_i, x_j) = \sigma_f \exp\left(-(x_i - x_j)^T W (x_i - x_j)\right) \quad (2)$$

where W is a diagonal matrix of hyperparameters which scale each dimension. With this kernel, the primary assumption about the underlying field is that the closer two locations are, the more similar their values are. A more complex kernel could be used, but as modeling is not the focus of this work, we restrict ourselves to this simple kernel.

4.2. EMSE in a Gaussian Process

For the results presented in this paper, we use EMSE in a Gaussian distribution as our objective function. We find that this objective function works well in practice, and it corresponds to the intuitive notion of reducing variance.

We denote the posterior covariance matrix over all considered locations after taking measurements at a set $\mathcal{A}$ of locations as $\Sigma_{\mathcal{A}}$. In the case where no measurements are taken, $\mathcal{A}$ is the empty set and the covariance matrix is simply the prior of the GP. We denote this prior covariance matrix as Σ_0. Calculating the posterior covariance matrix given a set of measurement locations and the kernel function in Equation (2) is straightforward; for the predictive equations we direct the reader to Rasmussen and Williams (2006).

Given the prior covariance matrix Σ_0 and the posterior covariance matrix for a set of measurement locations $\Sigma_{\mathcal{A}}$, we can now calculate the EMSE of that set of measurement locations,

$$\text{EMSE}(\mathcal{A}) = \frac{1}{N}[\text{tr}(\Sigma_0) - \text{tr}(\Sigma_{\mathcal{A}})]$$

In this equation, N is the total number of possible measurement locations, and is a constant. The function $\text{tr}(\cdot)$ is the trace of a matrix. The prior covariance matrix is also a constant (does not depend on the set of measurement locations visited by the robot). Since we aim to find a robot path which maximizes the EMSE of the visited measurement locations, we can ignore these constants.

5. Variants of IPP

The algorithm presented in this paper is motivated by two specific variants of IPP. The first is that of monitoring a time-varying field. If the change in the field is explicitly modeled, then the planner should take this into account when deciding what path will provide the most useful samples. The second variant is that of edge-based samples. For robots which sample as they move, the locations visited will be along the edges of the graph, not at the nodes themselves. In this section we describe these cases, and in Section 6 we explain how our algorithm handles them.

5.1. Time-varying objective functions

An important class of objective functions is composed of functions which depend on the time at which each node is visited. This could be the case, for example, if the goal is to

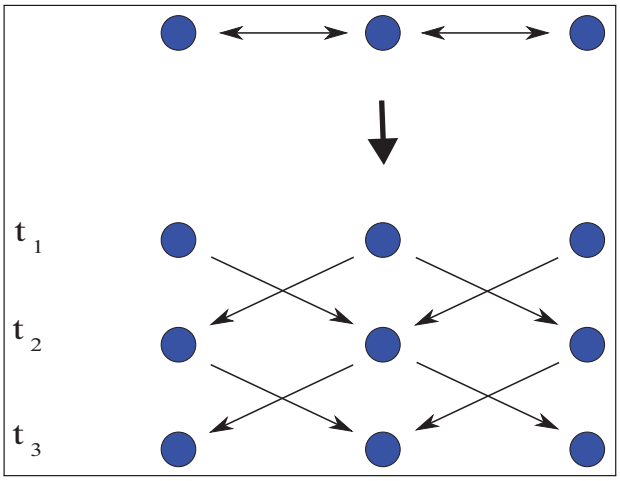

Fig. 1. How to convert a simple three-node graph into a time expanded graph which has copies of each node for each of three discrete timesteps.

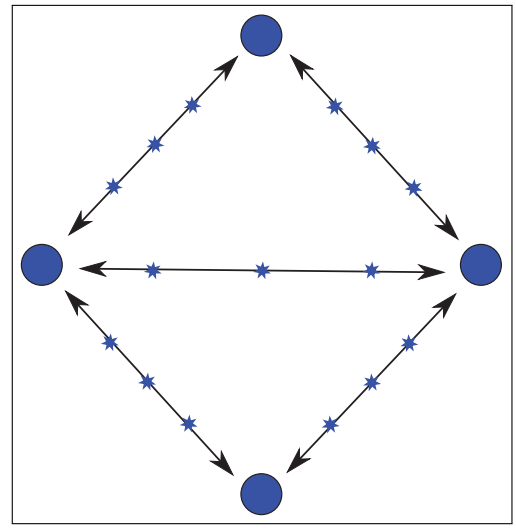

Fig. 2. A graph with three nodes and three sample points along each edge. These are locations at which a robot traversing that edge of the graph would take sensor measurements.

estimate a scalar field which changes with time. The value of taking a sample at a given location can then depend on the time at which it is taken. One approach to this problem is to expand the graph, making a copy of each node for each time at which it can be visited. For example, the top graph in Figure 1 can be expanded into the graph shown below it. In the expanded graph, time progresses downward. A path planning algorithm can then be used on the expanded graph, and the objective function can incorporate time by checking to see which copy of each node was visited. Unfortunately, this approach leads to a significant increase in the number of nodes in the graph, which can lead to an unreasonable running time increase for algorithms such as recursive greedy. Instead, our algorithm does path planning on the original graph, but works with time-varying objective functions.

Allowing time-varying objective functions is particularly important for long paths, or planning for multiple robots. If it is possible to traverse the area of interest several times, it becomes important to space out readings of nearby locations over time. The original recursive greedy algorithm makes no such distinction, and assigns the same value to measuring a location several times in quick succession as it does to making measurements of that location spaced out over time. Our algorithm allows the objective function to give the highest values to paths that sample the area of interest most effectively in both time and space.

5.2. *Edge-based samples*

Another important class of objective function is those which depend on the edges traversed, instead of (or in addition to) the nodes visited. This is true for sensors that sample continuously as the robot moves. In such cases, the samples taken may be much closer together than the waypoints in the path planning graph. The graph in Figure 2 gives an example of such a case, with the three samples taken along each edge marked by stars.

Edge-based samples can be incorporated by modifying the objective function to be defined on the edges traversed rather than the measurement locations visited. We handle this with a function that maps from edges to sample locations.

6. Algorithm

6.1. *Overview*

The original recursive greedy algorithm (Chekuri and Pal, 2005) provides an approximate solution to the submodular orienteering problem. This is the problem of finding a path from a node s to a node t in a graph G which maximizes a submodular function $f(V(P))$ of the nodes visited by the path. The recursive greedy algorithm works by recursively optimizing the gain obtained by adding a new partial path to the current set. At each level of recursion, all possible middle nodes for the path are tried, as are all possible ways to split the budget between the halves of the path. For each of these possibilities, the algorithm calls itself recursively to find the best path for the first half, then fixes that half and solves for the best path for the second half.

Owing to the recursive way in which it works, recursive greedy has an $O((2nB)^i T_f)$ running time, where n is the number of nodes in the graph, B is the maximum cost of the path, i is a maximum recursion depth chosen to be at least $1 + \lceil \log k \rceil$, where k is the number of nodes in the optimal path, and T_f is the maximum running time of the objective function. This running time, while subexponential, is still extremely sensitive to the number of nodes in the graph, and to the length of the path. The reason that recursive greedy

is attractive for IPP is that it provides an log-factor approximation guarantee for the solution it finds. We describe this more in Section 6.6.

At each step, recursive greedy keeps track of the nodes visited by parts of the path it has committed to, expressed by the function:

$$f_X(P) = f(V(P) \cup X) - f(X) \tag{3}$$

where $V(P)$ is the set of vertices that make up the path P, and X is the set of samples which have already been taken. In order to capture the starting time τ, we generalize this to

$$f_X(P, \tau) = f(h(P, \tau) \cup X) - f(X) \tag{4}$$

where we have replaced $V(P)$ with $h(P, \tau)$, a function which takes a path P and a starting time τ, and returns the set of samples that the robot would take. In the simplest case, we could choose $h(P, \tau) = V(P)$, in which case we get the original recursive greedy algorithm. By using other functions for $h(P, \tau)$, however, the algorithm can handle the time-varying and edge-based cases. We are able to prove that as long as $h(P, \tau)$ satisfies a simple property (described next), the approximation guarantee of recursive greedy still holds.

6.2. Requirements on $h(P, \tau)$

For a path P_1 that starts at time τ_1 and has length $\tau_2 - \tau_1$, and a path P_2 which starts at the end node of P_1 at time τ_2, we require that

$$h(P_1 \cdot P_2, \tau_1) = h(P_1, \tau_1) \cup h\ \ (P_2, \tau_2) \tag{5}$$

where $P_1 \cdot P_2$ is the concatenation of paths P_1 and P_2. This assumption is all that we need to preserve the approximation guarantee of the original recursive greedy algorithm.

6.3. How $h(P, \tau)$ expresses the edge-based sample case

As the robot moves, the path P consists of not only the set of nodes visited, but also the order in which they are visited. Owing to this, the set of edges visited is known, and it is simple to construct a function $h(P, \tau)$ which takes a path and returns the sample points which would be obtained along the edges. In this case, τ is not needed and can be ignored.

If we split the path of the robot into two parts, P_1 and P_2, the robot first takes the samples it would get as it traveled along the edges of P_1, and then collects the samples that it would get as it traveled along the edges of P_2. Owing to this, Equation (5) is trivially satisfied.

The recursive greedy algorithm assumes that the set of samples on which the objective function depends is the same as the set of nodes that the robot visits in the planning graph. If we wanted, we could add every sample location as a separate node in the planning graph, and do planning over this more complex graph.

Unfortunately, the computational complexity of the recursive greedy algorithm depends strongly on the number of nodes in the graph, and in this case it would quickly become intractable. Instead, we present a method which does path planning on the original graph, but computes the objective function over the set of samples that the robot would collect as it moved along each edge of its path. We show that the original approximation guarantee (see Theorem 1) holds.

6.4. How $h(P, \tau)$ expresses the time-varying case

To handle this case without increasing the computational complexity of the algorithm, we need to make the assumption that the edge costs in the graph G represent the time required to move from one node to the other.

Once we make this assumption, given that we know the start time, τ, of the path, it is possible to calculate the time at which each node in the path will be visited. Here $h(P, \tau)$ returns a set of (node, time) pairs such that the objective function can depend on the time at which each node is visited.

As in the edge-based case, if we split the path into two parts P_1 and P_2, the robot first collects the samples $h(P_1, \tau_1)$ as it travels along P_1, and then collects the samples $h(P_2, \tau_2)$ as it travels along P_2. Here τ_1 is the starting time of P_1, and τ_2 is the starting time of P_2, as required by Equation (5).

We could use recursive greedy to find the optimal path in the time-expanded graph from Figure 1, which would allow the value of our objective function to depend on the time at which the robot visited each node. The problem with this approach, as with using an expanded graph in the edge-based case, is that it greatly increases the number of nodes, which makes the running time much worse. If for every node we create B copies (for the B timesteps that the path will take), then the running time of recursive-greedy increases from $O((2nB)^i T_f)$ to $O((2nB^2)^i T_f)$. In our experience, this leads to impractically long running times in practice. Instead, we are able to do planning over the original graph, but map from the path in the original graph to a set of samples which are taken at a given time.

6.5. Pseudocode

In Algorithm 6.1, we give the pseudocode for the modified algorithm (GRG). In addition to using our modified f_X, the algorithm keeps track of the start and end times of the paths as it recurses, similar to the algorithm for recursive greedy with time windows in Chekuri and Pal (2005).

At each step, the algorithm looks for a path from node s to node t, starting at time τ_{start} and ending at time τ_{end}, to augment the set of samples X that have already been taken. We denote the length of the path from node s to node t

Algorithm 1 GRG $(s, t, \tau_{start}, \tau_{end}, X, i)$

```
if l(s, t, τ_start) > τ_end − τ_start then return Infeasible
P ⇐ s, t
if i = 0 then return P
m ⇐ −∞
for v ∈ V do
    for τ_start ≤ τ ≤ τ_end do
        P_1 ⇐ GRG(s, v, τ_start, τ, X, i − 1)
        P_2 ⇐ GRG(v, t, τ, τ_end, X ∪ h(P_1, τ_start), i − 1)
        if f_X(P_1 · P_2, τ_start) > m then
            P ⇐ P_1 · P_2
            m ⇐ f_X(P, τ_start)
return P
```

as $l(s, t, \tau_{start})$. If this length is greater than the remaining budget then this branch of the recursion is abandoned.

The parameter i keeps track of the depth of the recursion, so that the algorithm knows when to terminate. It tries every possible middle node v, and every possible middle time τ, and recurses to find the best first half and second half paths. During the recursion, the best value achieved so far is stored in the variable m.

Before evaluating the objective function, the algorithm applies a function $h(P, \tau)$ which maps the robot's path to the set of sample locations which will be visited. The set of sample locations depends on the time at which the robot traverses each edge in the graph.

6.6. Approximation guarantee

For monotonic, submodular set functions, recursive greedy is guaranteed to find a path whose objective value is at most logarithmically (in the number of nodes in the path) worse than the optimal path. More formally, it will always return a solution P such that $f_X(P) \geq \frac{f_X(P^*)}{\lceil 1+\log k \rceil}$, where P^* is the optimal solution (Chekuri and Pal, 2005).

Although we allow more general objective functions than in Chekuri and Pal (2005), the same approximation guarantee holds. This is very important, because global approximation guarantees are generally difficult to obtain for path planning with non-trivial objective functions.

Theorem 1. *For any objective function of the form $f(h(P, \tau))$ where $f(\cdot)$ is a monotonic submodular set function and $h(P, \tau)$ satisfies Equation* (5)*, GRG returns a solution P such that $f_X(P, \tau) \geq \frac{f_X(P^*, \tau)}{\lceil 1+\log k \rceil}$, where P^* is the optimal solution.*

A proof of this theorem is given in the appendix.

6.7. Running time

Because the recursion of the algorithm works on the original graph G, and does not depend on the function $h(\cdot)$, the running time is $O((2nB)^i T_f)$, as in the original algorithm. Although we treat the function $f(A)$ as a black box, it should be noted that typically the running time T_f of $f(h(P))$ will be longer if $h(P)$ returns more samples. For the variance reduction objective function which we use, the asymptotic running time is dominated by the Cholesky decomposition needed to calculate the posterior distribution. The Cholesky decomposition runs in $O(n_s^3)$, where n_s is the total number of possible samples. This brings the total running time of GRG to $O((2nB)^i n_s^3)$. For the original recursive greedy algorithm, $n_s = n$, because the samples and the nodes in the graph are the same. For choices of $h(P, \tau)$ which return more samples, there will be an increase in the running time, but it will be only a polynomial increase.

6.8. Equilateral graphs

As a side note, we mention that using graphs where all edges are of equal length leads to an additional speed up for both recursive greedy and for GRG. By applying a simple scaling factor, we can set all edge costs to be one. At each step of the recursive-greedy algorithm, all possible middle vertices and all possible allocations of B between the first and second half are tried. In the case where all edges have length one, we know that the length of the first half of the path is $\lceil k/2 \rceil$ and the length of the second half of the path is $B - \lceil k/2 \rceil$. Owing to this it is not necessary to try all possible values for the split at each step, significantly improving the running time. Instead of a running time of $O((2nB)^i T_f)$ we now have $O((2n)^i T_f)$, which is a significant improvement for large values of B (long paths). We could get additional (non-asymptotic) speedups by employing the branch and bound techniques used in Singh et al. (2009a)

It does not always make sense to use a graph in which all edges are of the same length. In cases where the mobility of the robot is approximately the same in all areas being considered, this is a good approximation. In cases where the mobility varies quite a bit over the area, it is a poor approximation and will result in paths with lengths that do not accurately represent the amount of time or power that are required to complete them. In the aquatic case, this approximation may be reasonable for an AUV which moves much faster than the current, but quite poor for slower AUVs in areas with strong currents. For a ground robot, the assumption would be good for cases where the terrain does not vary much in terms of navigability.

Although using graphs with equal length edges provides some advantages and we use it for the results in this paper, nothing in our algorithm requires such a graph. The GRG algorithm that we have presented and the accompanying approximation guarantee apply to general graphs.

6.9. Scalability and applicability to similar problems

The relatively high running time of GRG means that it does not scale to very large problems. In our experience, it is reasonable to use on graphs with up to a few dozen nodes.

Beyond that, the running time may be prohibitive. Because robots are actually exploring a continuous space, the complexity of the graph depends on how finely the space is discretized. The autonomous underwater and surface vehicles which our lab uses for environmental monitoring are typically controlled by giving them a fairly coarse sequence of spatial waypoints. For this reason we find it reasonable to use a fairly coarse graph for the planning problem. Robots which are controlled at more fine grained levels would need a graph with many more nodes to adequately represent their possible movements, and so GRG may not scale to those cases.

For GRG to be applicable to a problem, the problem must have several characteristics. First, the possible movements of the robot must be described as a graph of waypoints, and edges between them. Second, the start and end locations of the robot must be fixed. This is convenient when a field robot is being deployed and collected at known locations, but is not true for all problems. Finally, the objective function must be described as a function of the set of measurement locations visited, and there must be a mapping between robot paths and measurement locations which satisfies Equation (5). This would include cases where the Gaussian process model was replaced with another method of predicting covariance based on measurement locations, or where mutual information or entropy was used instead of EMSE.

As we prove in the appendix, GRG has an approximation guarantee for any objective function which is submodular in the set of measurement locations visited. Its empirical performance, however, will depend on the characteristics of the graph used and the exact objective function. In this work, we look at the empirical performance of GRG for IPP using fairly simple graphs with the EMSE objective function. We expect that it will also have good empirical performance on other objective functions, but have not yet performed experiments to verify this.

7. Results

The are many instance of IPP problems, each with different characteristics. Graphs can have different structure, different objective functions and underlying probabilistic models can be used, and the time budget can be varied. We have chosen three problems, each with different properties, to examine the performance of our algorithm. The first problem uses a triangular grid-based graph, and a simple GP with fixed hyperparameters. The second problem has a similar graph, but a much more complex Gaussian model using an empirical covariance matrix calculated from simulated ocean temperature data. The third problem again uses a simple GP, this time to model wireless signal strength on the surface of a lake. We learn the hyperparameters from an initial run, and then later collect data along all edges in the graph to test the actual variance reduction achieved when real samples are taken along the path chosen by our algorithm.

7.1. Finite horizon planning

For comparison, we have implemented a simple brute force planner with limited horizon. This planner works by trying all plans out to the fixed horizon, and choosing the best one. Then it fixes the first step of the planned path, and plans a new path starting at the next node. When an infinite horizon is used, this algorithm reduces to pure brute force search of all possible paths and will find the optimal solution. When a horizon of one is used, the algorithm becomes purely greedy, always choosing the next node which provides the most immediate gain in the objective function. As the horizon is increased, the running time of this planner increases exponentially.

Although intuition would suggest that increasing the horizon of this planner will result in finding a better path, this is not always true. Paths which look good over a slightly longer horizon may prove to be worse over an even longer horizon. Unless an infinite horizon is used, the exact objective value achieved by the planner depends somewhat on luck. Unfortunately, even for the simple graphs in our examples, using an infinite horizon would require an infeasible amount of computation time. Still, we find it useful to plan paths using multiple different horizons, to provide multiple different plans which we can compare against the solution found by GRG.

7.2. Simple example

We first test the algorithm on one of the simplest possible cases; a stationary GP with a simple covariance function. The graph used is shown in Figure 3. We use a radial basis function (RBF) kernel with one scaling parameter. All edges are undirected and of unit length. This example assumes that the robot takes measurements along the graph edges as it moves, but that the field it is monitoring does not vary significantly over time. The locations at which the robot would sample as it travels along each edge are marked by red triangles. Because we can take advantage of the ability of GRG to handle edge-based samples, the running time does not depend on the number of samples on each edge (except when adding samples causes the objective function itself to be harder to compute).

The path chosen by GRG is shown in Figure 4. Because it plans ahead, it is able to choose a nicely spaced out path while observing the budget constraint. In contrast, the path chosen by a purely greedy algorithm (a planner that looks one step ahead and greedily chooses the edge which gives the most gain) is shown in Figure 5. It quickly moves to the middle of the graph, and ends up not having enough budget remaining to space out the rest of the path.

A purely greedy planner is fairly naive, so we have also implemented a brute force planner for longer horizons. A

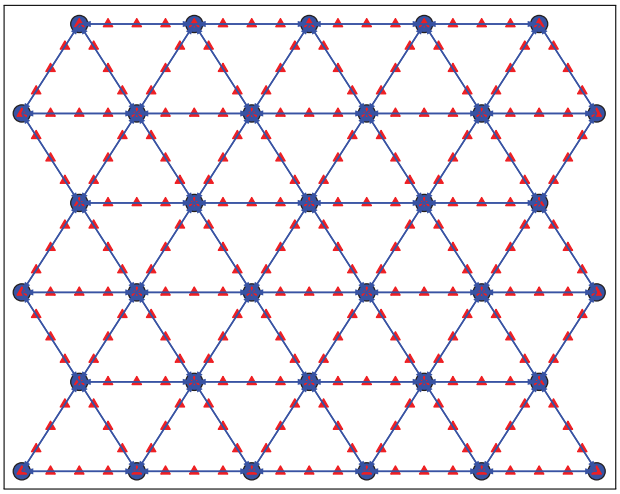

Fig. 3. Graph used as an example problem. There are 33 nodes (blue circles) connected by undirected edges. Sample positions along each edge are marked by red triangles.

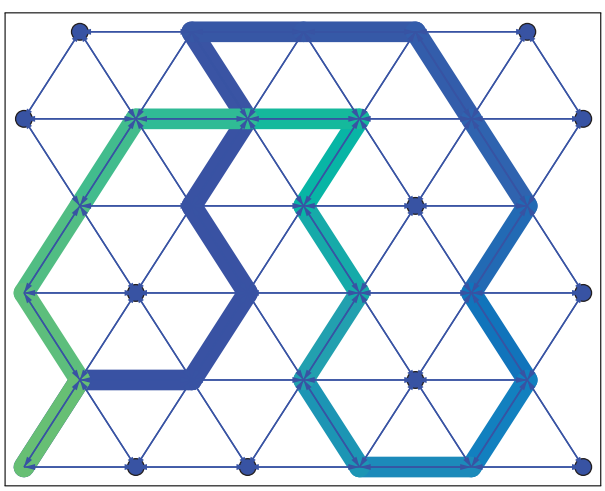

Fig. 4. Path produced by GRG algorithm. The path starts and ends at the bottom left node. Shading added only to make it easier to follow the path as it crosses itself.

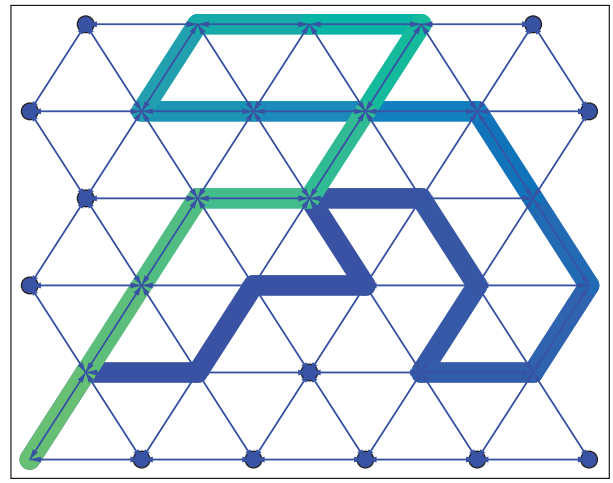

Fig. 5. Path produced by brute force planner with a horizon of one (greedy). The path starts and ends at the bottom left node. Shading added only to make it easier to follow the path as it crosses itself.

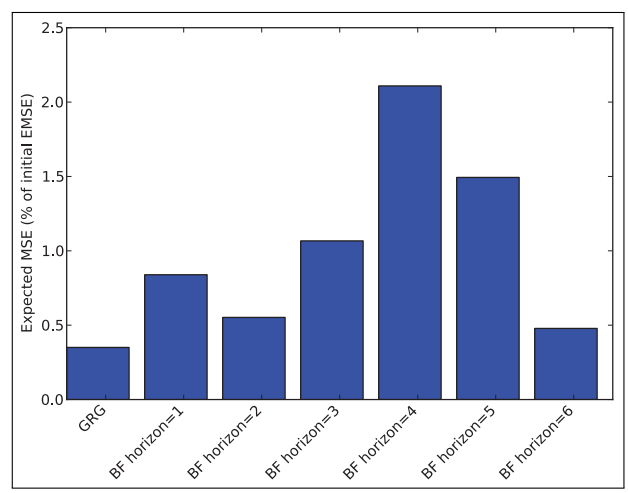

Fig. 6. EMSE values for the paths produced by GRG, and for brute force (BF) planning with various horizons. Lower EMSE values are better.

brute force planner with a horizon plans head a fixed number of steps, and picks the best path for those steps. Then it adds the first edge to its solution path, and replans. The EMSE achieved by GRG is compared with brute force planners with various horizons in Figure 6. GRG outperforms the finite horizon approach even up to a horizon of six. It may seem somewhat counterintuitive that the performance does not strictly improve as the horizon increases, this does sometimes happen. Looking slightly further ahead can cause the algorithm to make a choice which is actually worse.

The computation times for both GRG and the brute force method with various horizons is shown in Figure 7. These results are given as the runtime, in seconds, for an unoptimized python implementation on a standard PC. As expected, brute force with a horizon of one (pure greedy) is by far the fastest. As the horizon is increased, the runtime of brute force increases exponentially, and running it with a horizon of six takes longer than GRG. Looking back at Figure 6, we can see that even though brute force with a horizon of six takes longer to compute than GRG, it does not achieve as low of an EMSE. The running times and EMSE values achieved will depend on the graph and objective function of each specific problem, but the general takeaway is that the choice of algorithm will depend on the importance of finding a path that is close to optimal versus the amount of time and computation power available.

7.3. *ROMS simulated dataset results*

The problem examined in Section 7.2 was chosen for its simplicity, and used a simple GP kernel. In such a case, IPP is basically a coverage problem, since the correlation

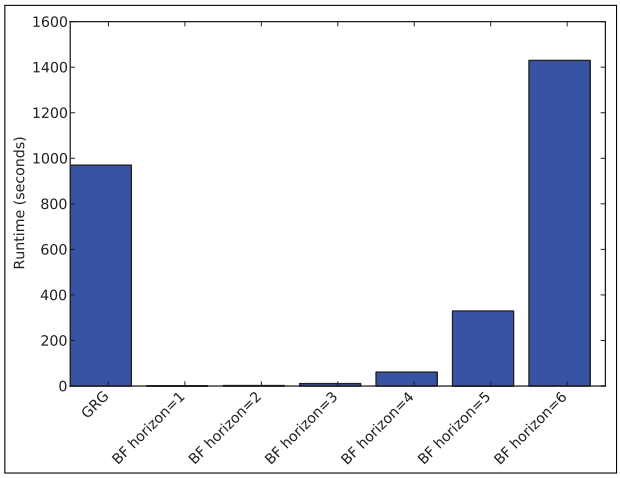

Fig. 7. Runtimes in seconds for GRG and for brute force (BF) planning with various horizons. Tests were done using single threaded implementations in Python on a Pentium Core i5 processor.

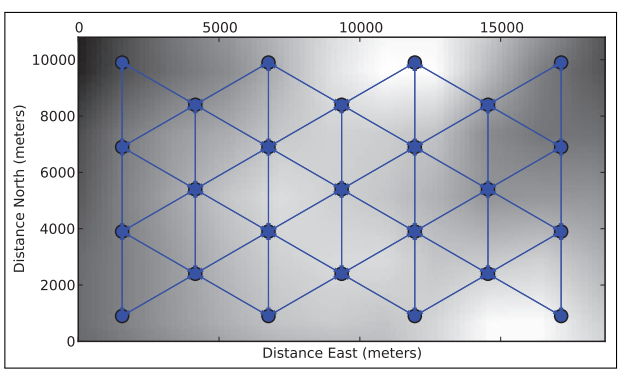

Fig. 8. Graph used for the results in Section 7.3, overlaid on a snapshot from one moment in time of the temperatures in the simulated dataset.

between samples depends only on the distance between them. IPP becomes truly interesting when we have a complicated and non-stationary kernel function.

To show this, we have created a simulated dataset using data from the Regional Ocean Modeling System (ROMS) (Shchepetkin and McWilliams, 2005; Li et al., 2008). This system uses a computational model of the ocean combined with sensor inputs. It predicts currents, temperature, and salinity for the area of the Pacific Ocean off of the Southern Californian coast. ROMS provides predicted current, temperature, and salinity values for each 2.2×2.2 km^2 patch of the ocean, at various depths, for each hour. ROMS calculations are released each day at 13:00 GMT, and provide predictions for the next 72 hours.

For this problem we choose water temperature as our quantity of interest. We will be planning a path for an AUV that has a time budget of 48 hours. Because the water temperature varies significantly over this length of time, we explicitly model change over time in our GP.

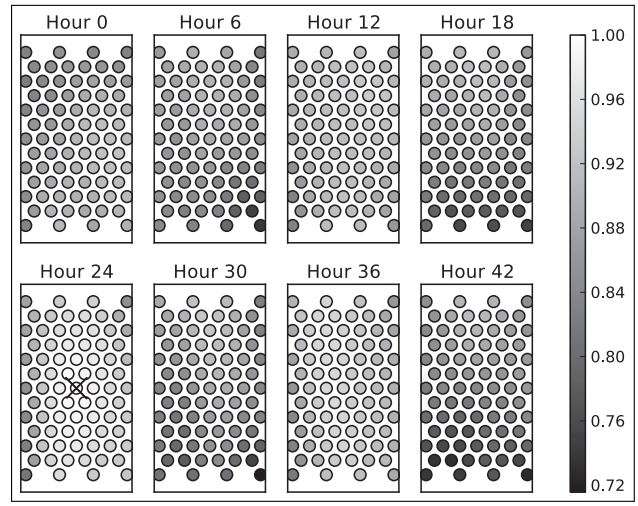

Fig. 9. Correlations shown between a reference point and all other sample points on the edges. The reference point is marked with an 'x' (at hour 24). White fill indicates points that are highly correlated with the reference point, and black fill indicates points which are less correlated.

The area we chose for our simulated experiment is North of Santa Catalina island, off the coast of Southern California. A rectangular "region of interest" was chosen, defined by the latitude and longitude of the corners. For this experiment, we planned on a two-dimensional plane, the slice of ocean at a depth of 35 m.

To define our IPP problem, we construct the graph shown in Figure 8 over the region of interest. Here we have overlaid the graph on a snapshot of the temperatures in the region at one instant in time. We have constructed the graph for a hypothetical AUV that moves at a velocity of 1 km/h (currents are ignored). Each edge in the graph is 3 km long, and so has a cost of 3 hours. For the results in this section, we give the AUV a time budget of 48 hours, which allows it to traverse 16 edges in the graph. We assume that the robot samples at the nodes, and once as it traverses an edge.

After defining the spatial and temporal locations of all possible samples that the robot can take, we use ROMS data from 50 consecutive days to calculate the empirical covariance of all of those sample locations. We then calculate the empirical covariance of the temperature predictions. A plot of the calculated covariance between one example point in the middle of the region and all other points at each time can be seen in Figure 9. Interestingly, the correlation shows a periodic trend with time.

The path chosen by GRG for the ROMS simulated dataset is shown in Figure 10. Interestingly, it does not appear to achieve good spatial coverage as did the solution to the simple problem in Section 7.2. We attribute this to the varying correlations over time.

As we can see in Figure 11, GRG achieves better EMSE than a brute force planner, even when the planning horizon is extended to six steps. Runtimes of the algorithms are

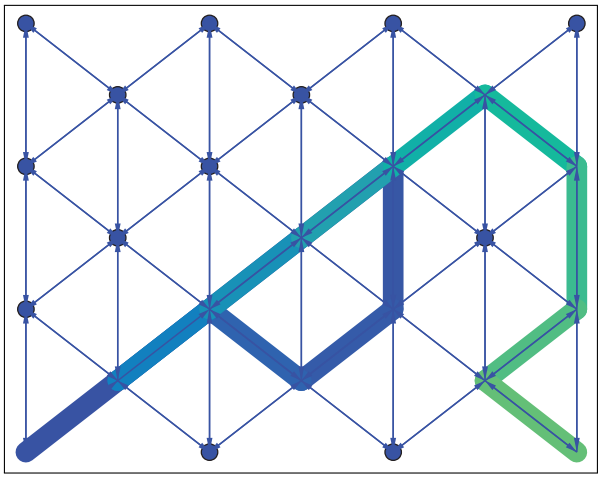

Fig. 10. Path produced by GRG algorithm on the ROMS simulated dataset. The path starts and ends at the bottom left node. Shading added only to make it easier to follow the path as it crosses itself.

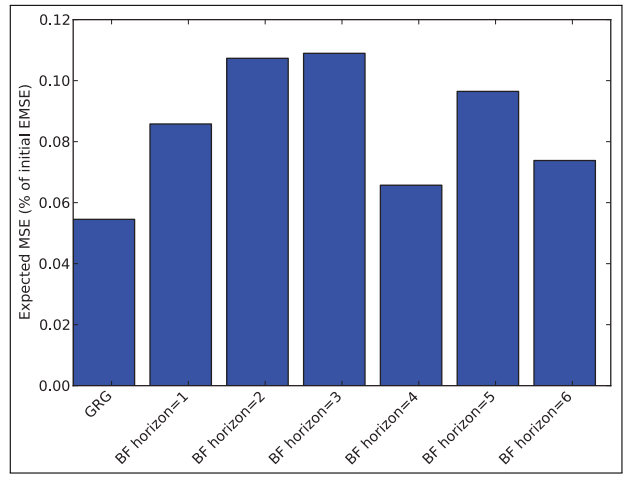

Fig. 11. EMSE values for the ROMS simulated dataset. Shows EMSE for the paths produced by GRG, and for brute force (BF) planning with various horizons. Lower EMSE values are better.

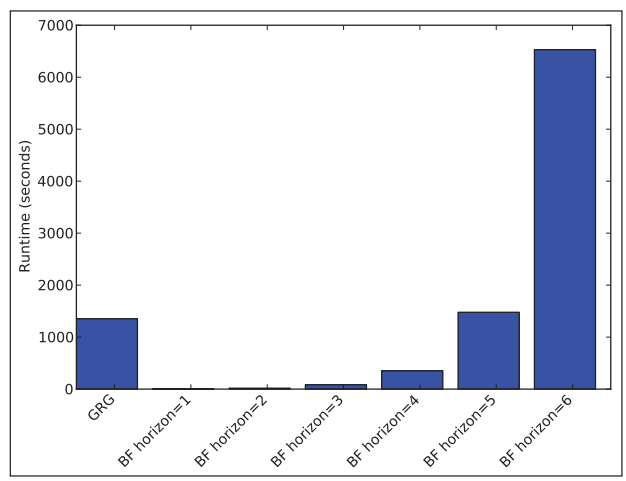

Fig. 12. Runtimes in seconds for GRG and for brute force (BF) planning with various horizons on the ROMS simulated dataset. Tests were done using single threaded implementations in Python on an Intel Core i5 processor.

shown in Figure 12. For horizon lengths greater than five, brute force planning takes significantly longer than GRG.

7.4. Field results

The two previous example problems we have presented so far show only the EMSE, given the model. Owing to unmodeled effects and stochasticity, EMSE may not correspond perfectly to mean square error (MSE) once real measurements have been taken and used to make predictions. To examine this, we deployed the ASV shown in Figure 13b at a lake in Southern California. More information on the hardware of the platform can be found in Heidarsson and Sukhatme (2011).

An overhead view of the experiment location can is given in Figure 13a. Because of the relatively small scale of the area we are operating in ($\sim$200 m $\times$ $\sim$300 m), the quantities which the aquatic sensors on the boat can measure do not appear to vary significantly. Instead, we build a GP model of the wifi signal strength from the wireless router which we use to communicate with the boat.

A grid shaped waypoint graph is constructed over the region in which the boat can safely operate. Ignoring a scaling factor, we assume all edges have a traversal cost of one. Using a short initial run, hyperparameters are learned for an isotropic, stationary GP with an RBF kernel. Then, using this kernel, paths are planned to minimize EMSE. We have run our algorithm for two cases: one where the robot must start and finish its path at the lower middle node and one where it must start and finish its path at the top left node. It is desirable for the start and end waypoints to be the same because it is easiest to deploy and retrieve the vehicle at the same location on the shore.

The EMSE values achieved by each of the path planners for the lake signal strength dataset is shown in Figure 14. GRG achieves similar or lower than expected MSE values than the finite horizon planners. Some of the finite horizon planners are able to stumble upon good solutions due to the simplicity of the graph, but none perform significantly better than GRG. It is interesting to note that increasing the horizon of the brute force planner does not always improve the resulting path. Although this is counterintuitive, it does happen. Looking further ahead can cause the planner to choose a path which appears better, but turns out to be worse over an even longer horizon. The only guarantee is that with a full horizon, the brute force algorithm will find an optimal path. Unfortunately this requires an impractical amount of computation time.

To evaluate the prediction accuracy that would be achieved after collecting measurements along the planned path and making predictions, the ASV was used to collect wireless signal strength data along all edges on the graph (shown in Figure 15). We then used the data which would

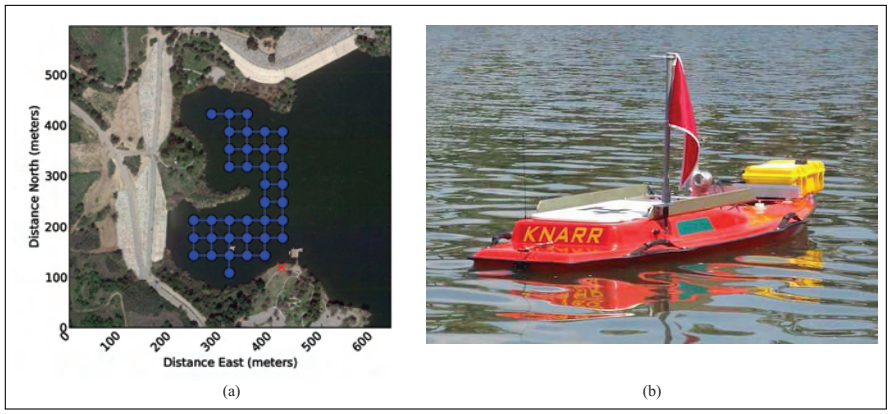

Fig. 13. (a) Waypoint graph overlaid on an overhead view of the location for the field experiment. The location of the wireless router is marked by a red "x". (b) ASV used to collect wireless signal strength data for Section 7.4. More information about the platform can be found in Heidarsson and Sukhatme (2011).

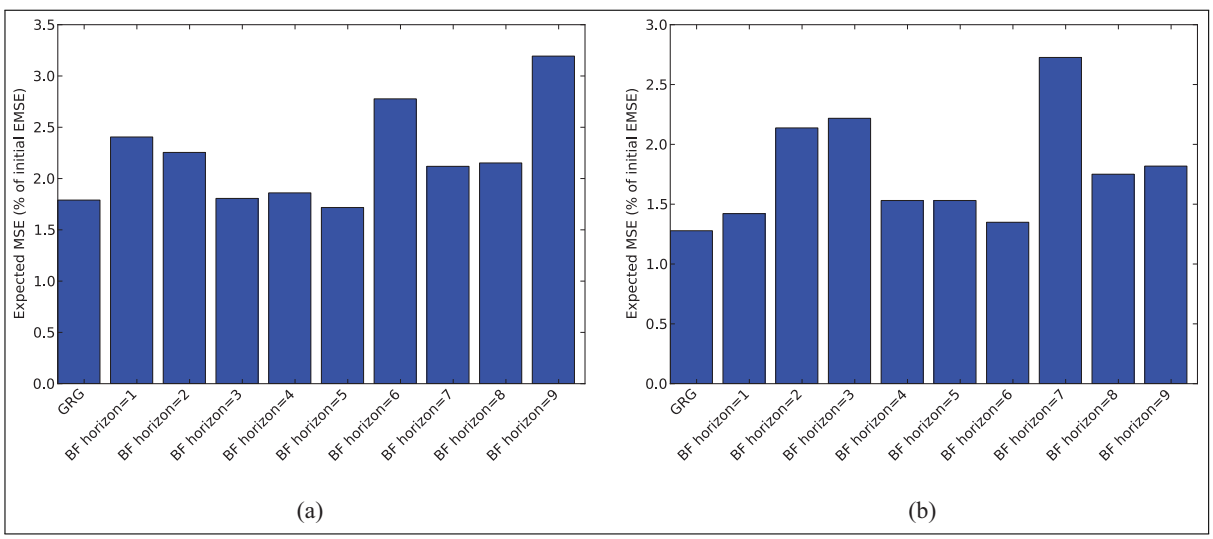

Fig. 14. EMSE values achieved by the path planning algorithms for the lake wireless signal strength experiment: (a) the results when planning paths that start and end at the bottom middle node; (b) the results when planning paths that start and end at the top left node. Lower EMSE values are better.

be collected along each planned path to predict the values of the signal strength measurements taken along all of the edges. The resulting MSE values are shown in Figure 16a. The MSEs are considerable higher than the corresponding EMSE, partly attributable to the fact that we are using a GP with a simple covariance function which is far from a perfect model of signal strength. Still, the results seem to show that the path produced by GRG is reasonable.

Looking at the algorithm runtimes in Figure 17a we can see that running the brute force planner with a horizon longer than eight steps takes significantly longer than running GRG. Owing to the shape of the graph (two main areas connected by a narrow strip), a finite horizon planner starting at one end might not see far enough ahead to know to save enough time to explore the other region sufficiently.

7.5. Discussion of results

The results from these three example problems show that there are cases where GRG provides a good balance between computational efficiency and solution quality. Given a particular spatiotemporal monitoring problem, a number of factors will determine the choice of algorithm. For extremely small graphs, or when computation time is not an issue, brute force could be used to find the optimal path. For large graphs, or when computation is extremely limited, a heuristic solution such as the greedy algorithm may provide a "good enough" solution. For in-between cases, GRG may be the best choice.

An important characteristic of the spatiotemporal monitoring problem is that the monitoring path chosen for the

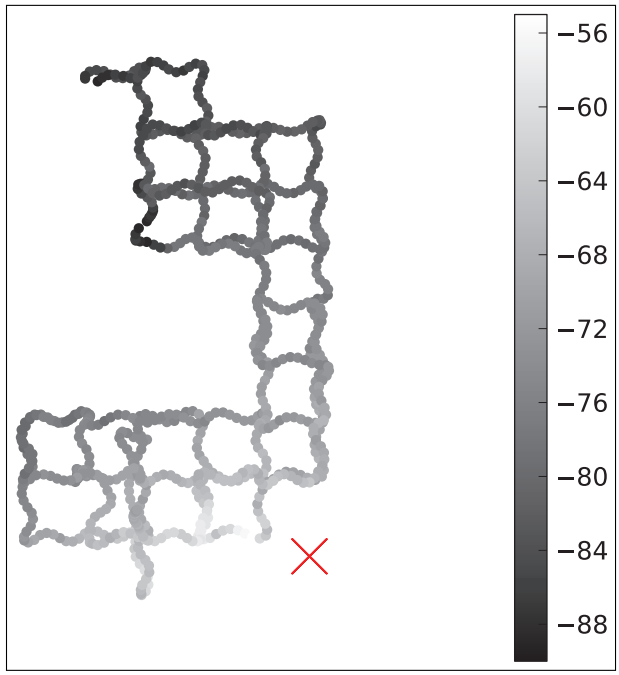

Fig. 15. Wireless signal strength data (in decibels) collected to evaluate the performance of the planners. The location of the base station is marked near the bottom with a red "x". Because of wind, the ASV does not perfectly track the edges of the graph.

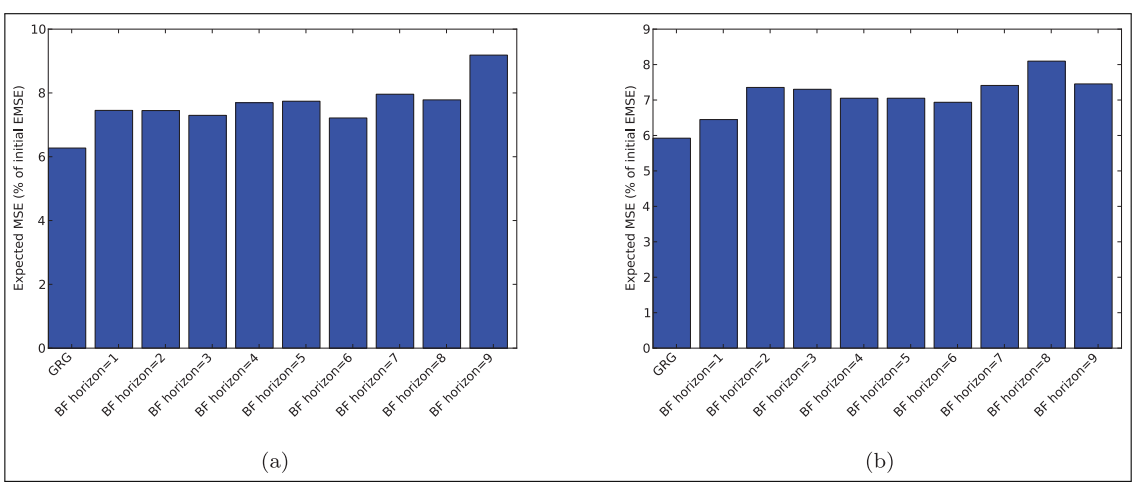

Fig. 16. Actual MSE values achieved by the path planning algorithms for the lake wireless signal strength experiment: (a) the results when planning paths that start and end at the bottom middle node; (b) the results when planning paths that start and end at the top left node. Lower EMSE values are better.

robot depends on the model chosen for the underlying phenomenon. In Section 7.4, we saw that even though GRG chose paths which the GP model predicted would result in low EMSE, the true MSE is still unexpectedly high. This seems to be due mostly to the aspects of the phenomena being measured (wireless signal strength) that cannot be adequately handled by the simple GP model used. For this work we are interested in analyzing the path planning algorithm, and so we accept this drawback. In the future, however, we are interested in finding more realistic domain specific models, and testing our algorithm on them.

8. Conclusion

Submodular objective functions are powerful enough to express realistic notions of sensing quality (such as mutual information), but still allow for an approximate path planning algorithm that runs in subexponential time. Previous works have used the recursive greedy submodular orienteering approximation algorithm to do path planning, but were limited to objective functions that are submodular in the set of nodes visited. As we have shown, this can be a very limiting assumption, and rules out two very important

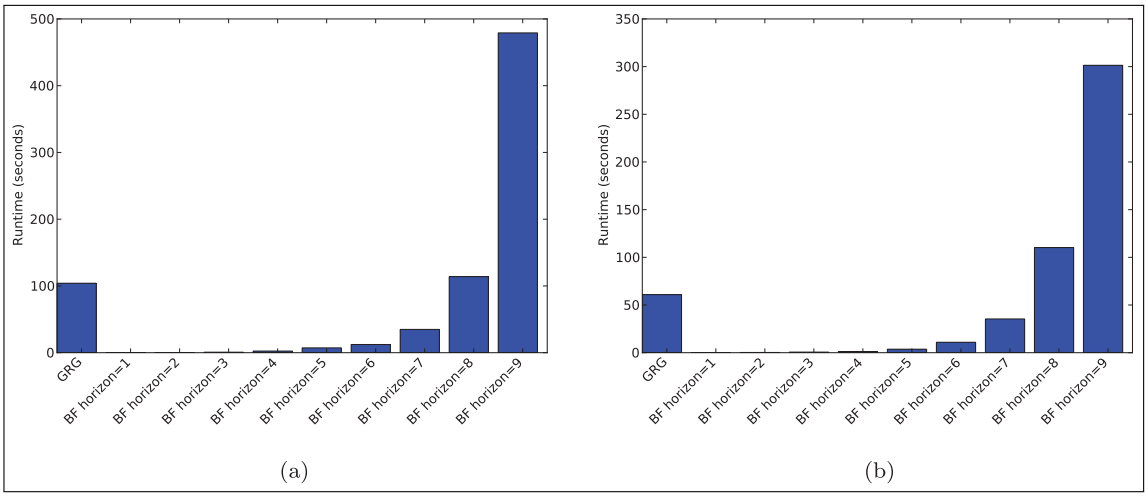

Fig. 17. Runtimes in seconds for GRG and for brute force (BF) planning with various horizons for the lake signal strength experiment: (a) the results when planning paths that start and end at the bottom middle node; (b) the results when planning paths that start and end at the top left node. Tests were done using single threaded implementations in Python on an Intel Core i5 processor.

cases: edge-based samples and time-varying models. We have presented a method, GRG, which incorporates more general objective functions that are submodular not in the set of vertices visited, but in the set of samples which will be taken as the robot traverses the path.

Using these objective functions, we have shown how to do path planning for the time-varying case, and for the case in which samples are taken along the edges of the graph as the robot moves. In addition, we have provided a proof that the original approximation guarantee holds.

Although we have focused on the provable approximation guarantee in this work, there are many other promising directions for path planning using submodular objective functions. In the future, we plan to use the method for multiple robot path planning. Allowing the objective function to take into account the time at which the samples are taken should make it possible to plan multi-robot paths that obtain good coverage in both time and space. We also plan to address ways of speeding up the path planning, so that it can be used on larger graphs.

Funding

This work was supported in part by the US National Science Foundation under the Center for Embedded Networked Sensing (grant number CCR-0120778), by the NSF (grant numbers CNS-1035866 and IIS-1125015), by the ONR MURI program (grant numbers N00014-09-1-1031 and N00014-08-1-0693), by the ONR SoA program and by a gift from the Okawa Foundation.

References

Binney J, Krause A and Sukhatme GS (2010) Informative path planning for an autonomous underwater vehicle. In: *Proceedings of IEEE International Conference on Robotics and Automation*.

Blum A, Chawla S, Karger DR, Lane T, Meyerson A and Minkoff M (2007) Approximation algorithms for orienteering and discounted-reward TSP. *SIAM Journal on Computing* 37(2): 653–670.

Chekuri C and Pal M (2005) A recursive greedy algorithm for walks in directed graphs. In: *46th Annual IEEE Symposium on Foundations of Computer Science, 2005*. pp. 245–253. DOI: 10.1109/SFCS.2005.9.

Das A and Kempe D (2008) Algorithms for subset selection in linear regression. In: *Proceedings of the 40th Annual ACM Symposium on Theory of Computing*, Victoria, BC. New York: ACM Press, pp. 45–54. DOI: 10.1145/1374376.1374384.

Heidarsson HK and Sukhatme GS (2011) Obstacle detection from overhead imagery using self-supervised learning for autonomous surface vehicles. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems*, San Francisco, CA, pp. 3160–3165.

Hollinger G and Singh S (2008) Proofs and experiments in scalable, near-optimal search by multiple robots. *Proceedings of Robotics: Science and Systems IV*, Zurich, Switzerland.

Krause A, Guestrin C, Gupta A and Kleinberg J (2006) Near-optimal sensor placements: maximizing information while minimizing communication cost. In: *The Fifth International Conference on Information Processing in Sensor Networks*. Nashville, TN, pp. 2–10. DOI: 10.1109/IPSN.2006.244031.

Krause A, Singh A and Guestrin C (2008) Near-optimal sensor placements in gaussian processes: Theory, efficient algorithms and empirical studies. *Journal of Machine Learning Research* 9: 235–284.

LaValle SM (2006) *Planning Algorithms*. Cambridge: Cambridge University Press.

Li Z, Chao Y, McWilliams JC and Ide K (2008) A three-dimensional variational data assimilation scheme for the regional ocean modeling system. *Journal of Atmospheric and Oceanic Technology* 25(11): 2074–2090.

Lilienthal A and Duckett T (2004) Building gas concentration gridmaps with a mobile robot. *Robotics and Autonomous Systems* 48(1): 3–16. DOI: 10.1016/j.robot.2004.05.002.

Meliou A, Krause A, Guestrin C and Hellerstein JM (2007) Nonmyopic informative path planning in spatio-temporal models. In: *Proceedings of the 22nd National Conference on Artificial Intelligence*, Vol. 1, Vancouver, BC. AAAI Press, pp. 602–607.

Narendra PM and Fukunaga K (1977) A branch and bound algorithm for feature subset selection. *IEEE Transactions on Computers* 26(9): 917–922. DOI: 10.1109/TC.1977.1674939.

Rasmussen CE and Williams CKI (2006) *Gaussian Processes for Machine Learning*. Cambridge, MA: MIT Press.

Rooker MN and Birk A (2007) Multi-robot exploration under the constraints of wireless networking. *Control Engineering Practice* 15(4): 435–445. DOI: 10.1016/j.conengprac.2006.08.007.

Russell S and Norvig P (2002) *Artificial Intelligence: A Modern Approach* (2nd edn). Englewood Cliffs, NJ: Prentice Hall.

Shchepetkin AF and McWilliams JC (2005) The regional oceanic modeling system (ROMS): a split-explicit, free-surface, topography-following-coordinate oceanic model. *Ocean Modelling* 9(4): 347–404. DOI: 10.1016/j.ocemod.2004.08.002.

Singh A, Krause A, Guestrin C and Kaiser W (2009a) Efficient informative sensing using multiple robots. *Journal of Artificial Intelligence Research* 34: 707–755. DOI: 10.1613/jair.2735.

Singh A, Krause A and Kaiser WJ (2009b) Nonmyopic adaptive informative path planning for multiple robots. In: *Proceedings of the 21st International Joint Conference on Artifical Intelligence*. San Francisco, CA: Morgan Kaufmann, pp. 1843–1850.

Somol P, Pudil P and Kittler J (2004) Fast branch & bound algorithms for optimal feature selection. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 26(7): 900–912. DOI: 10.1109/TPAMI.2004.28.

Stachniss C, Plagemann C, Lilienthal A and Burgard W (2009) Gas distribution modeling using sparse Gaussian process mixture models. In *Robotics:Science and Systems IV*. Cambridge, MA: MIT Press, pp. 310–317.

Vasudevan S, Ramos F, Nettleton E and Durrant-Whyte H (2009) Gaussian process modeling of large-scale terrain. *Journal of Field Robotics* 26(10): 812–840. DOI: 10.1002/rob.20309.

Yilmaz N, Evangelinos C, Lermusiaux P and Patrikalakis N (2008) Path planning of autonomous underwater vehicles for adaptive sampling using mixed integer linear programming. *IEEE Journal of Oceanic Engineering* 33(4): 522–537. DOI: 10.1109/JOE.2008.2002105.

Zhang B (2008) *Adaptive Sampling with a Robotic Sensor Network*. PhD thesis, University of Southern California.

Proof of approximation guarantee

This proof closely follows that of Chekuri and Pal (2005), but has been modified to include the function $h(P,\tau)$. Like the original proof, it works by induction on k, the length of an optimal path. We keep most of the notation the same; the important difference is that we redefined f_X in Equation (4), and have modified the proof to account for the new, more general definition.

A few words on notation: P^* is the optimal path; P_1^* and P_2^* are the first and second halves of the optimal path, respectively; P_1 starts at time τ_1, and P_2 starts at time τ_2; P_1 and P_2 are the first and second halves of the path that recursive greedy chooses, respectively. Because the algorithm tries every possible middle node and every possible middle time at each step when it splits the path, we can always choose a P_1 which starts and ends at the same nodes and at the same time as P_1^*. The same is true for P_2.

The base case, when $k = 1$, is simply $P = (s,t)$. In this case, the algorithm tries the path (s,t) and so $f_X(P,\tau) = f_X(P^*,\tau)$.

By induction, we are able to assume

$$f_X(P_1,\tau_1) \geq \frac{1}{\lceil \log k \rceil} f_X(P_1^*,\tau_1) \tag{6}$$

$$f_{X'}(P_2,\tau_2) \geq \frac{1}{\lceil \log k \rceil} f_{X'}(P_2^*,\tau_2) \tag{7}$$

where $X' = X \cup h(P_1,\tau_1)$. Now in the inductive step, we must show that when we join these two paths together into a longer path, the approximation guarantee is preserved:

$$\begin{aligned} f_{X'}(P_2^*,\tau_2) &= f(h(P_2^*,\tau_2) \cup (h(P_1,\tau_1) \cup X)) \\ &\quad - f(h(P_1,\tau_1) \cup X) && (8) \\ &= f(h(P_1 \cdot P_2^*,\tau_1) \cup X) - f(h(P_1,\tau_1) \cup X) && (9) \\ &= f_X(P_1 \cdot P_2^*,\tau_1) + f(X) \\ &\quad - (f_X(P_1,\tau_1) + f(X)) && (10) \\ &= f_X(P_1 \cdot P_2^*,\tau_1) - f_X(P_1,\tau_1) && (11) \end{aligned}$$

This is true because we know from Equation (5) that $h(P_1,\tau_1) \cup h(P_2^*,\tau_2) = h(P_1 \cdot P_2^*,\tau_1)$. Substituting this back into Equation (7) gives

$$\begin{aligned} f_{X'}(P_2,\tau_2) &\geq \frac{1}{\lceil \log k \rceil}(f_X(P_1 \cdot P_2^*,\tau_1) - f_X(P_1,\tau_1)) && (12) \\ &\geq \frac{1}{\lceil \log k \rceil}(f_X(P_2^*,\tau_2) - f_X(P,\tau_1)) && (13) \end{aligned}$$

This is because of the monotonicity of $f(\cdot)$. Specifically,

$$\begin{aligned} & f_X(P_1 \cdot P_2^*,\tau_1) - f_X(P_1,\tau_1) \\ &\quad = f(h(P_1 \cdot P_2^*,\tau_1) \cup X) - f(X) \\ &\qquad - (f(h(P_1,\tau_1) \cup X) - f(X)) && (14) \\ &\quad \geq f(h(P_2^*,\tau_2) \cup X) - f(X) \\ &\qquad - (f(h(P,\tau_1) \cup X) - f(X)) && (15) \\ &\quad = f_X(P_2^*,\tau_2) - f_X(P,\tau_1) && (16) \end{aligned}$$

We are able to go from Equation (14) to Equation (15) because $f(\cdot)$ is monotonic, and we know from Equation (5) that $h(P_2^*,\tau_2) \subset h(P_1 \cdot P_2^*,\tau_1)$, $h(P,\tau_1) \supset h(P_1,\tau_1)$.

Adding Equation (6) and Equation (13) gives

$$\begin{aligned} & f_X(P_1,\tau_1) + f_{X'}(P_2,\tau_2) \\ &\quad \geq \frac{1}{\lceil \log k \rceil}(f_X(P_1^*,\tau_1) + f_X(P_2^*,\tau_2) - f_X(P,\tau_1)) \end{aligned} \tag{17}$$

$$f_X(P,\tau_1) \geq \frac{1}{\lceil \log k \rceil}(f_X(P^*,\tau_1) - f_X(P,\tau_1)) \tag{18}$$

because of submodularity. Specifically, we use the fact that $f_X(P_1^*, \tau_1) + f_X(P_2^*, \tau_2) \geq f_X(P^*, \tau_1)$.

Finally, we rearrange terms in Equation (18) to complete the inductive step of the proof.

$$f_X(P, \tau_1) \geq \frac{1}{1 + \lceil logk \rceil} f_X(P^*, \tau_1) \tag{19}$$